

Doubly Linked List – Creation of Empty List And Time Complexity.

Program :

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    struct DLL *next;
    struct DLL *prev;
}Node;

Node *head;

void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

→ *Node, is a structure variable i. e. the name given to struct DLL achieved through typedef keyword, gives new name to existing data types, here for struct DLL. Struct is a composite data type which contains group of several variables or different data types , grouped under a single name or single type.*

Node is given name to struct DLL ,As shown below:

```
typedef struct DLL
{...} Node;
```



We can also write it as:

```
typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;
```

And , now :

```
Node *head;
```

Without , typedef we can rewrite it as:

```
struct Node
{
    int data;
    Node *next;
    Node *prev;
};

Node *head;
```

OR

```
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};

struct Node *head;
```

Now ,

```
Node *head;
```

is a global variable, which are accessible to any part of the program, and as we need to start from head and keep track of it , its easy to perform by making the variable global otherwise with local scope we have to create each seperate head variable while calling each modular functions.

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;

Node *head;
```

The Rule: Zero – Initialization

In C ++, any variable declared at the global scope (outside of all functions) has static storage duration. The C ++ standard guarantees that variables with static storage duration are zero – initialized by the compiler if you do not explicitly initialize them.

For a pointer, "zero – initialization" means it is automatically set to a null pointer.

Node * head

Address Head

ff21

NULLPTR

*And , then * head initialized to null pointer(explicitly).*

```
void createEmptyList()
{
    head = nullptr;
}
```



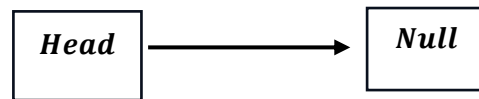
*Note: It is just explicitly assigning Node * head to Nullptr, which is already assigned with NULLPTR , since Node * head is global pointer variable, following :The Zero Initialization Rule.*

C + + version 11 → Null Pointer vs MACRO NULL

In C + +03 and earlier versions, there was no dedicated null pointer literal. Developers often used the constant 0 or macros like NULL to represent null pointers. The problem with this approach is that an integer value of 0 could accidentally be assigned to a pointer variable, leading to unexpected behavior and potential crashes.

In essence, nullptr is a special literal value that represents the absence of a valid pointer in C + +. It was introduced in the C + +11 standard to enhance type safety and improve code clarity. The nullptr, however, is a distinct type, std :: nullptr_t, designed explicitly for pointer absence. By using nullptr, the compiler can enforce type safety and distinguish between an intentionally null pointer and an integer value. This reduces the potential for errors caused by accidental assignment of integers to pointers. The nullptr enhances code readability and self – documentation. Its explicit representation of a null pointer makes the code's intent clearer. Using nullptr makes the code more readable and less prone to errors. This helps prevent subtle bugs that could arise from accidentally assigning the wrong value to a pointer. The nullptr is a dedicated keyword that represents the null pointer value.

The nullptr is a keyword introduced in C + +11 to represent a null pointer. It is used to indicate that a pointer does not point to any object or function. The nullptr has its own type, std :: nullptr_t, which can be implicitly converted to any pointer type but not to integral types. This helps prevent errors where a null pointer might be confused with an integer. We can use nullptr wherever you need to assign or compare a pointer to a null value. Hence Creation of Empty Linked List , according to Linked List representation:



Head points to Null, is creation of empty linked list.

Time Complexity of the program

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data; → O(1)
    DLL *next; → O(1)
    DLL *prev; → O(1)
}Node;

Node *head; → O(1)

void createEmptyList()
{
    head = nullptr; → O(1)
}

int main()
{
    createEmptyList(); → O(1)
    return 0; → O(1)
}
```

To discuss about time complexity of the program module of creation of

empty linked list :

→ Declaration of data , pointer variable * next, pointer variable * prev takes $O(1)$ i. e. constant time complexity.

→ Renaming of struct DLL to Node takes $O(1)$ i. e. constant time complexity.

→ Function call takes constant time complexity i. e. $O(1)$, including initialization of global Head pointer variable to NULL takes constant amount of time.

→ Returns 0, which is another constant – time operation takes $O(1)$.

Hence , total amount of time complexity : $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$ i. e. constant amount of time.

<i>Doubly Linked List</i>	<i>Time Complexity</i>
<i>Creation of Empty List</i>	<i>$O(1)$</i>
